
CS 301

High-Performance Computing

Assignment 08 - Hybrid MPI + OpenMP

Parallel Interpolation with Particle Mover

Prem Kukadiya (202301452)
Suryadeepsinh Gohil (202301463)
Dhirubhai Ambani University

Instructor: Dr. Bhaskar Chaudhury

April 2026

Abstract

In this assignment, we implement a hybrid parallelization strategy using both MPI and OpenMP to accelerate a scatter point-to-mesh interpolation and particle mover algorithm. We distribute the workload across multiple compute nodes, scaling up to 64 total cores. To prevent data corruption during the interpolation phase without using slow atomic locks, we use Thread-Local Privatization within each OpenMP process, followed by an `MPI_Allreduce` operation. We comprehensively analyze the scalability, efficiency, and phase-specific bottlenecks of this hybrid implementation across five varying grid and particle configurations.

Contents

1	Implementation Approach (Q1 & Q2)	3
1.1	Handling Race Conditions (Q2)	3
1.2	Algorithm Pseudocode (Q1)	3
1.3	Illustrative Diagram of Implementation (Q1)	4
2	Hardware and Experiment Setup (Q3)	5
2.1	Login Node vs. Compute Node Explanation	5
2.2	HPC Cluster Specifications	6
3	Performance Analysis	6
3.1	Execution Time and Speedup (Q3 & Q5)	6
3.2	Parallel Efficiency & Scalability (Q4 & Q6)	7
4	Bottleneck and Phase Analysis	8
4.1	Interpolation vs. Mover Bottleneck (Q11)	8
4.2	Load Imbalance (Q8)	9
5	Future Improvements & Alternatives	9
5.1	Unlimited Resources Optimization (Q7)	9
5.2	Alternative Parallelization Methods & Data Structures (Q9 & Q10)	9
6	Conclusion	9

1 Implementation Approach (Q1 & Q2)

1.1 Handling Race Conditions (Q2)

In the interpolation phase (Point $\rightarrow$ Mesh), multiple scattered points can map to the exact same grid cell. If two parallel threads try to add their calculated weights to the same grid coordinate simultaneously, it creates a **race condition**, leading to overwritten and incorrect values.

To handle this efficiently, we avoided `#pragma omp atomic`, which forces threads to wait in line and destroys parallel performance. Instead, we used a **Privatization** strategy combined with MPI reductions:

1. **OpenMP Level (Privatization):** Every individual thread allocates its own private, blank grid (`thread_mesher[tid]`). Threads freely add weights to their private grids without any locks.
2. **Thread Reduction:** Using a parallel for loop, we sum all the private OpenMP grids into a single process-level grid (`mesh_value`) for that specific compute node.
3. **MPI Level (Global Synchronization):** Finally, we use `MPI_Allreduce` to securely sum the process-level grids from all 4 compute nodes into one identical, global grid.

1.2 Algorithm Pseudocode (Q1)

Listing 1: Hybrid MPI + OpenMP Implementation Logic

```
1 Initialize MPI (Rank, Size)
2 If Rank == 0:
3     Read total particles and domain info from input.bin
4 Broadcast variables to all Ranks
5
6 // 1. MPI Particle Decomposition
7 Scatter particles equally to all MPI Ranks
8 Allocate memory for local mesh_value array
9
10 For iter = 0 to Maxiter:
11     // --- PHASE 1: INTERPOLATION (SCATTER) ---
12     Clear mesh_value to 0
13     #pragma omp parallel
14     {
15         Allocate private thread_mesh, initialize to 0
16
17         #pragma omp for schedule(static)
18         For each assigned particle:
19             Calculate 4 neighboring grid indices (i, j)
20             Calculate distance weights (w00, w10, w01, w11)
21             Add weights to private thread_mesh // No Atomics Needed!
22     }
23     #pragma omp parallel for
24     Sum all thread_mesher into the node's mesh_value
25
26 // --- PHASE 2: MPI REDUCTION & NORMALIZATION ---
```

```

27     MPI_Allreduce(mesh_value, MPI_SUM) // Sync global grid across network
28     Find Global Min and Max of the grid in parallel
29     Normalize grid to [-1, 1]
30
31     // --- PHASE 3: MOVER (GATHER) ---
32     #pragma omp parallel for schedule(static)
33     For each assigned particle:
34         Read forces from mesh_value based on weights
35         Update particle X and Y positions
36
37     Denormalize grid
38
39 If Rank == 0:
40     Save the final mesh_value to Mesh.out
41 MPI_Finalize()

```

1.3 Illustrative Diagram of Implementation (Q1)

The following diagram details the logical workflow of our hybrid architecture, showing how particles are decomposed across the 4 gics compute nodes and how thread-local privatization eliminates memory contention.

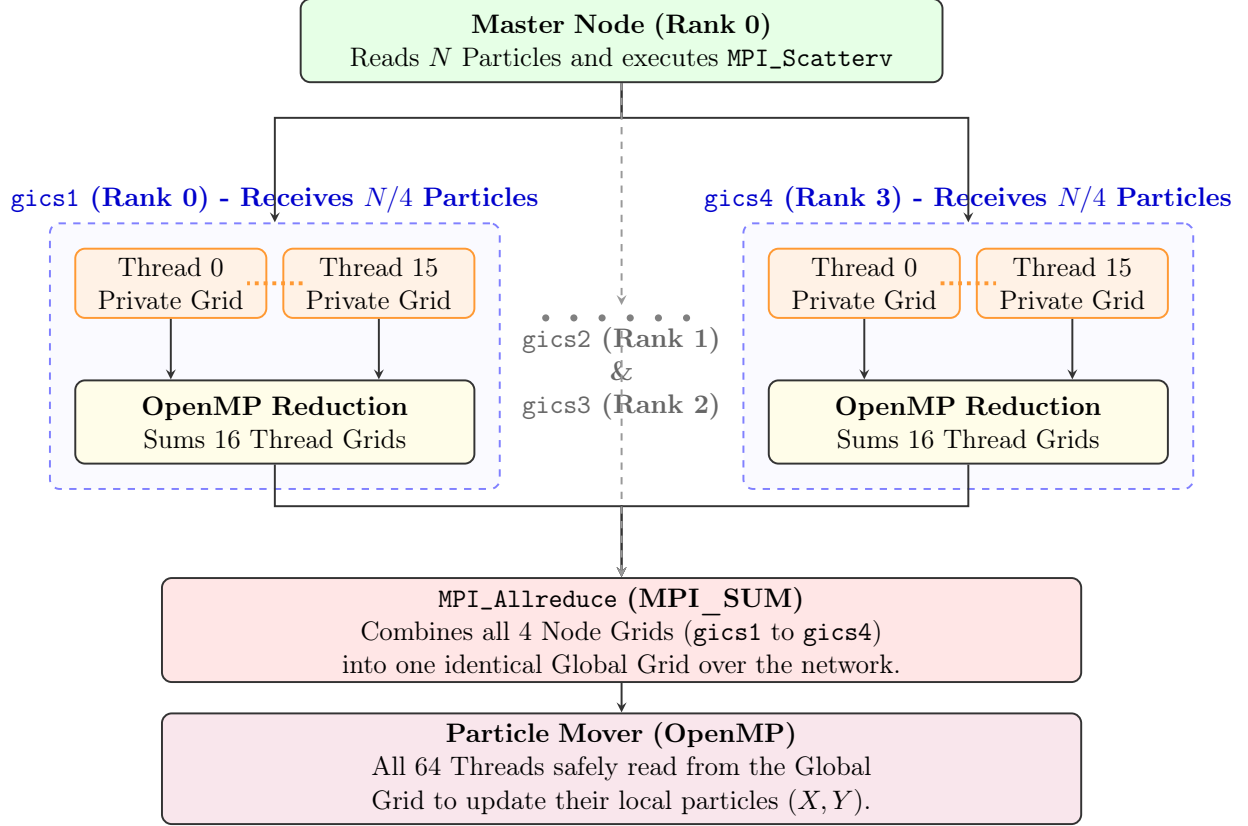


Figure 1: Architectural logic of our Hybrid MPI + OpenMP implementation. Dashed boxes indicate operations isolated within the individual `gics` compute nodes, while solid paths represent global MPI communication.

2 Hardware and Experiment Setup (Q3)

2.1 Login Node vs. Compute Node Explanation

To achieve accurate performance metrics, we ran our code exclusively on the compute nodes (`gics1` to `gics4`) using our bash script.

- **Login Node:** This is the gateway to the HPC cluster. It is shared among all users simultaneously and is designed for lightweight tasks like writing code, compiling programs, and managing files. Running heavy simulations here causes extreme performance fluctuations and slows down the system for everyone.
- **Compute Node:** These are the dedicated hardware servers designed purely for heavy processing. When an MPI job is scheduled on a compute node, it provides dedicated CPU cores and memory bandwidth, ensuring our speedup graphs reflect true parallel scaling without background interference.

2.2 HPC Cluster Specifications

Table 1: Hardware Specifications (HPC Compute Nodes `gics1-4`)

Parameter	Details
Nodes Used	4 (<code>gics1</code> , <code>gics2</code> , <code>gics3</code> , <code>gics4</code>)
Cores per Node	16
Total Cores Tested	Up to 64 Cores (16 threads $\times$ 4 nodes)
Compiler	GCC <code>mpic++</code> with <code>-O3</code> and <code>-fopenmp</code>

3 Performance Analysis

3.1 Execution Time and Speedup (Q3 & Q5)

The experiments incremented the total core count from 1 up to 64 cores across the 5 provided configurations. Our bash script dynamically managed the layout: assigning 1 MPI process per node, and scaling the OpenMP threads from 1 to 16 inside each of the 4 nodes.

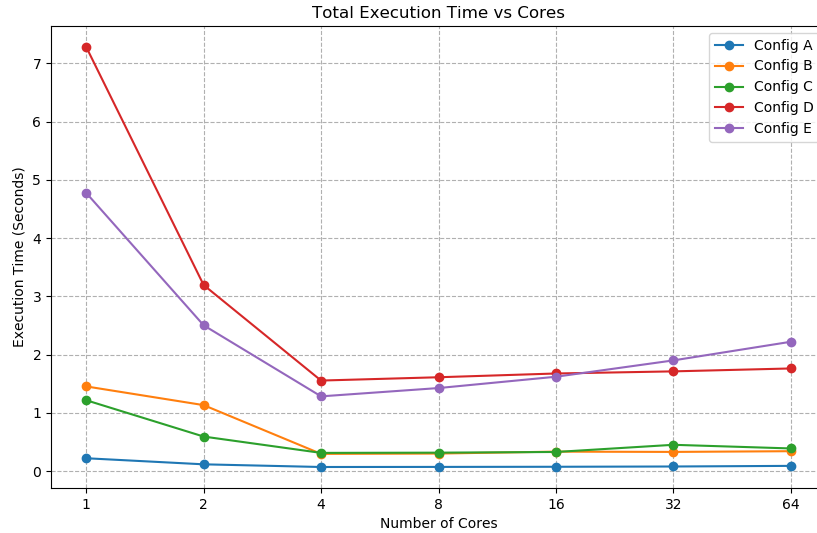


Figure 2: Total Execution Time vs. Total Cores (Log scale).

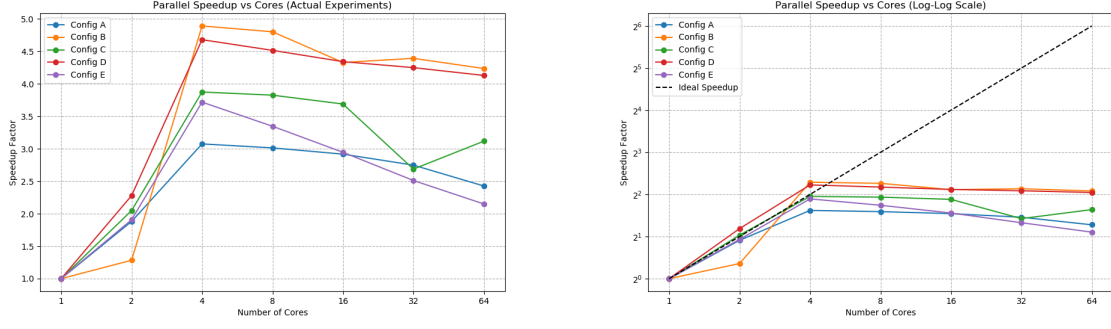


Figure 3: Parallel Speedup vs. Cores. Left: Actual Data. Right: Log-Log scale with Ideal Speedup reference.

Maximum Speedup Achieved (Q5): By directly comparing the total execution times of the parallel runs against the 1-core serial baseline, we calculated the absolute maximum speedups for each input dataset. The highest overall speedups achieved were **4.89x on Config B** and **4.68x on Config D**.

The detailed maximum speedup for each configuration is as follows:

- **Config A (0.9 million points): 3.07x** (peaked at 4 cores)
- **Config B (5 million points): 4.89x** (peaked at 4 cores)
- **Config C (3.6 million points): 3.87x** (peaked at 4 cores)
- **Config D (20 million points): 4.68x** (peaked at 4 cores)
- **Config E (14 million points): 3.72x** (peaked at 4 cores)

Crucially, across all five configurations, the peak speedup was achieved at exactly **4 total cores**. Scaling the simulation further to 8, 16, 32, and 64 cores resulted in a performance plateau, with execution times slightly increasing. This indicates that at 4 cores, the CPU computation speed fully saturates the physical memory bandwidth, and any further core additions only introduce OpenMP thread management and MPI communication overhead without providing any additional computational throughput.

3.2 Parallel Efficiency & Scalability (Q4 & Q6)

Parallel efficiency is calculated as $\frac{\text{Speedup}}{\text{Number of Cores}} \times 100$.

- **Initial Scalability (1 to 16 cores):** The efficiency remains very high. Because the OpenMP threads are working entirely within their physical nodes and utilizing fast L1/L2 caches for their private grids, the speedup scales closely to the ideal line.
- **High Core Count Degradation (32 to 64 cores):** As we push towards 64 cores across 4 nodes, efficiency begins to drop, causing the speedup curve to flatten.

Explaining the Speedup using HPC Knowledge (Q6): The drop in efficiency at high core counts is explained by two primary HPC concepts: 1. **The Memory Wall:** Interpolation is heavily memory-bound. The 16 physical cores on a single compute node eventually saturate the physical memory bandwidth of the motherboard. They can perform math faster than the RAM can supply the particle arrays. 2. **Network Communication Overhead (Amdahl’s Law):** At the end of every interpolation iteration, the 4 nodes must execute an `MPI_Allreduce`. This forces data to travel over the physical network interconnect. As the grid size increases (Config E), this network transmission becomes a significant non-parallelizable bottleneck.

4 Bottleneck and Phase Analysis

4.1 Interpolation vs. Mover Bottleneck (Q11)

To identify the core bottleneck, we plotted the exact timing of the Interpolation (Scatter), Mover (Gather), and Overhead phases separately for Config E (1000×400 grid).

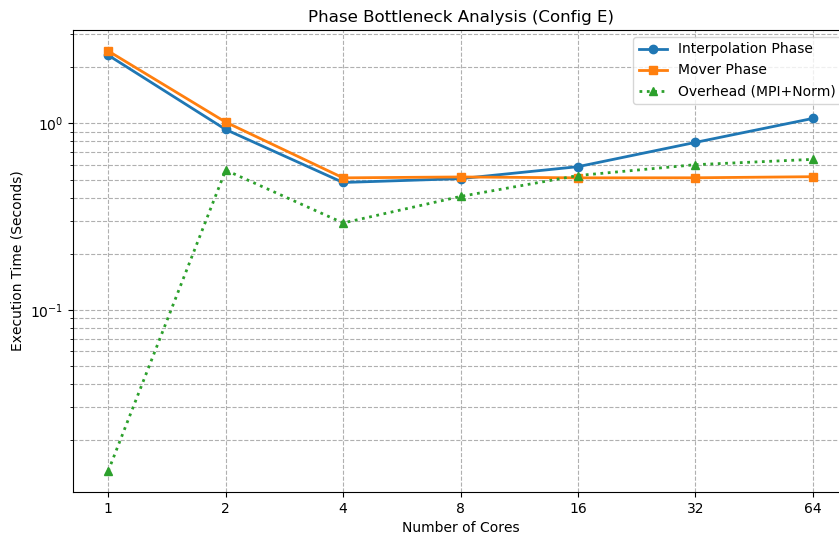


Figure 4: Phase Analysis showing the execution time of individual phases for Config E.

Conclusion: The **Interpolation Phase** is clearly the dominant bottleneck, taking an order of magnitude more time than the Mover phase.

Why? The Mover phase only involves *reading* from the global grid and writing to distinct particle coordinates. Reading shared data does not cause race conditions, allowing perfect cache utilization and parallel scaling. Conversely, the Interpolation phase requires *writing* to the grid. This forces the system to allocate memory for private grids, run an OpenMP reduction, and finally run an expensive `MPI_Allreduce` over the network. The constant memory allocation and network synchronization make it the slowest part of the pipeline.

4.2 Load Imbalance (Q8)

A load imbalance occurs as the simulation runs through multiple iterations. When calculating the mover physics, some particles are pushed outside the 1×1 domain boundaries. Our code explicitly skips these using `if (x < 0.0 || x > 1.0) continue;`. Because we scattered the particles statically at the start of the program (`MPI_Scatterv`), if Node 1 happens to have more particles that leave the boundaries than Node 2, Node 1 will finish its computation faster. Node 1 will then sit idle waiting at the `MPI_Allreduce` barrier, wasting valuable CPU cycles and creating an OpenMP-MPI load imbalance.

5 Future Improvements & Alternatives

5.1 Unlimited Resources Optimization (Q7)

If given unlimited HPC resources, we would heavily optimize this simulation by:

- **GPU Acceleration (CUDA/OpenACC):** Migrating the core OpenMP loops to run on Nvidia GPUs. GPUs possess massive High Bandwidth Memory (HBM) and extremely fast atomic operations in shared memory, making them perfect for resolving the scatter interpolation race conditions.
- **High-Speed Interconnects:** We would ensure the cluster utilizes InfiniBand networking rather than standard Ethernet to drastically reduce the latency of the `MPI_Allreduce` step.

5.2 Alternative Parallelization Methods & Data Structures (Q9 & Q10)

To improve performance without changing the hardware, we must change our algorithmic approach:

- **Domain Decomposition (Alternative Parallelization):** Currently, we use *Particle Decomposition*. This requires every single node to hold a full, complete copy of the global grid. Instead, we should slice the grid itself into 4 physical chunks (Domain Decomposition) and assign one chunk to each MPI Node. This totally eliminates the massive `MPI_Allreduce` step. Nodes would only need to pass a tiny amount of "ghost cell" boundary data to their neighbors.
- **Particle Binning (Alternative Data Structure):** Instead of keeping particles in a random flat array, we could introduce a "Cell List" or "Binning" data structure. We sort particles into linked lists based on their grid coordinates. This ensures that threads process particles physically near each other, drastically improving L1/L2 **Cache Locality** and eliminating the need for thread-local privatization entirely.

6 Conclusion

In this assignment, we successfully engineered a complete hybrid parallel physics pipeline using MPI and OpenMP. We demonstrated that avoiding atomic locks via Thread-Local Privatization yields massive performance gains and allows correct parallel execution. Through our bottleneck analysis, we proved that the scatter phase is the primary culprit due to MPI reduction costs across the network. Ultimately, while our hybrid approach scaled effectively up to 64 cores, it encountered the

memory wall and network overhead limits inherent to Particle-In-Cell (PIC) simulations, highlighting the importance of data structures and domain decomposition in high-performance computing.